

DSA0601 Data Handling and Visualization

Slot A

CO2 – Assessment Test 1 (AT1)

Chart Construction Test

Question 1 – Distribution Diagnosis using ECDF and Q-Q Plot

Given Data

Response time (ms):

118, 121, 125, 127, 129, 131, 134, 136, 138, 141, 144, 147, 151, 156, 162, 171, 184, 205, 249, 318

Task 1. Construct an ECDF and a Normal Q-Q Plot

R Program

```
# Response time data
```

```
response_time <- c(
```

```
  118, 121, 125, 127, 129,
```

```
  131, 134, 136, 138, 141,
```

```
  144, 147, 151, 156, 162,
```

```
  171, 184, 205, 249, 318
```

```
)
```

```
# Set plotting area for two graphs
```

```
par(mfrow = c(1, 2))
```

```
# -----
```

```
# ECDF Plot
```

```
# -----
```

```
plot(  
  ecdf(response_time),  
  main = "ECDF of Response Time",  
  xlab = "Response Time (ms)",  
  ylab = "Cumulative Probability",  
  verticals = TRUE,  
  do.points = TRUE  
)
```

```
# -----  
# Normal Q-Q Plot  
# -----
```

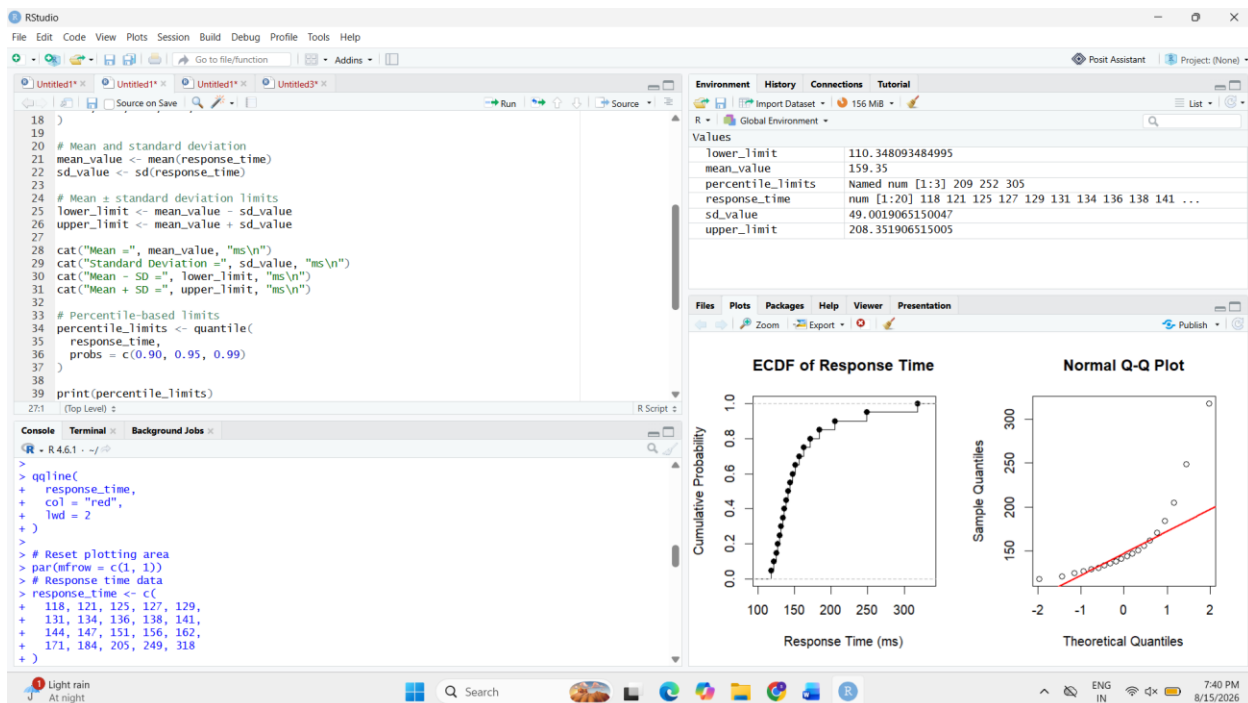
```
qqnorm(  
  response_time,  
  main = "Normal Q-Q Plot",  
  xlab = "Theoretical Quantiles",  
  ylab = "Sample Quantiles"  
)
```

```
qqline(  
  response_time,  
  col = "red",  
  lwd = 2  
)
```

```
# Reset plotting area
```

```
par(mfrow = c(1, 1))
```

Visualization



The **ECDF** displays how the cumulative proportion of observations increases with response time. The **Q-Q plot** compares the observed response times with the theoretical quantiles of a normal distribution.

Task 2. Use Both Plots to Evaluate the Normality Claim

ECDF Interpretation

The ECDF shows that most of the observations are concentrated between approximately 118 ms and 184 ms. After this region, the response times extend to 205 ms, 249 ms, and 318 ms.

This extended upper tail indicates that the observations are not distributed symmetrically around the centre. The distribution therefore shows evidence of right skewness.

Normal Q-Q Plot Interpretation

If the data were approximately normally distributed, the points in the Q-Q plot would lie reasonably close to the straight reference line.

For this dataset, the lower and middle observations may follow the reference line reasonably well, but the high response-time observations increasingly deviate from it.

In particular, 249 ms and 318 ms are expected to produce substantial deviation in the upper tail.

Task 3. Identify Evidence of Skewness and/or Extreme Observations

The dataset contains clear evidence of right skewness.

Most observations lie between 118 ms and 184 ms, whereas the final observations increase substantially:

- 184 ms
- 205 ms
- 249 ms
- 318 ms

The value 318 ms is particularly extreme compared with the main concentration of observations.

The observations 249 ms and 318 ms form a long upper tail, which is the main evidence of positive skewness.

Therefore:

Distribution: Right-skewed

Extreme observations: 249 ms and 318 ms

Most extreme observation: 318 ms

The mean can also be affected strongly by these high observations, making it less representative of the typical response time.

Task 4. Recommend Percentile-Based Limits or Mean $\pm$ Standard-Deviation Limits

Recommended approach: Percentile-based limits

Percentile-based limits are recommended rather than mean $\pm$ standard-deviation limits.

The reason is that the dataset is right-skewed and contains extreme upper-tail observations. Mean and standard deviation are sensitive to such observations.

For example, a high observation such as 318 ms can increase both the mean and standard deviation. Consequently, a limit based on:

Mean $\pm$ Standard Deviation

may not represent the actual distribution of typical response times accurately.

A percentile-based approach is more suitable because it directly describes the position of observations within the observed distribution.

For example:

- **50th percentile** → typical/median response time
- **90th percentile** → high response-time threshold
- **95th percentile** → suitable upper monitoring threshold

R Code for Percentile Calculation

```
# Calculate important percentiles
```

```
quantile(  
  response_time,  
  probs = c(0.50, 0.90, 0.95, 0.99)  
)
```

Optional Comparison with Mean $\pm$ SD

```
# Mean
```

```
mean(response_time)
```

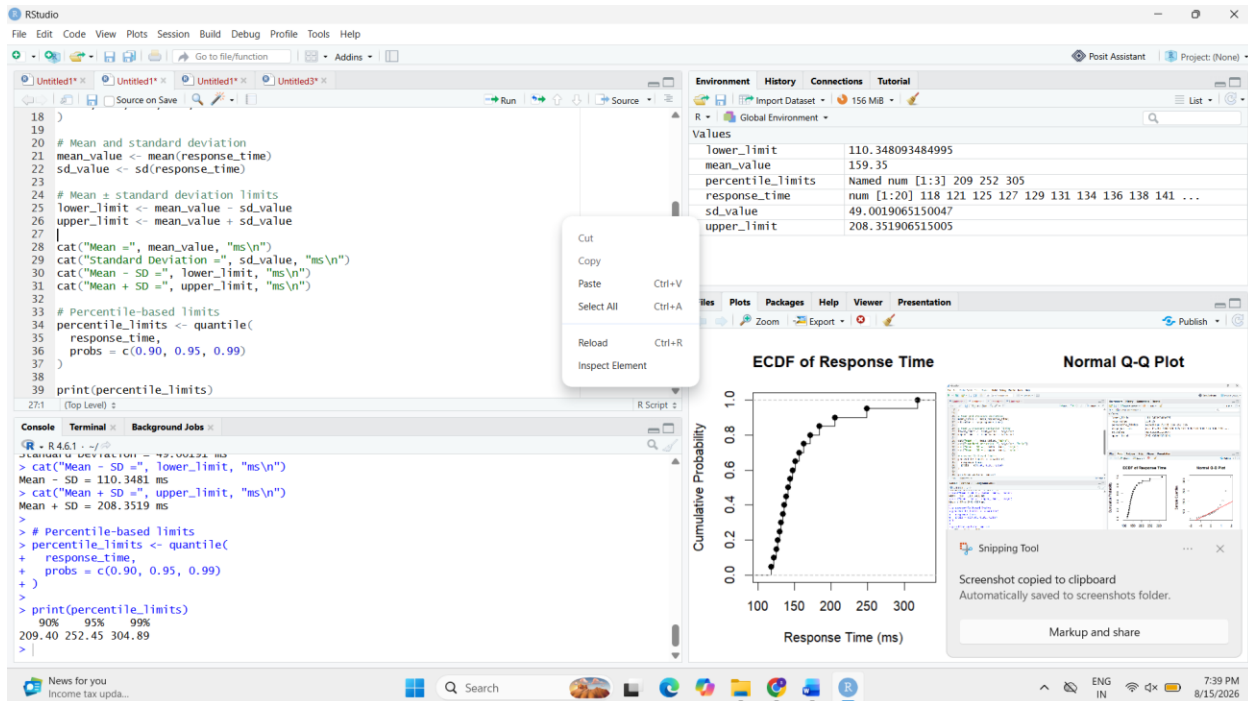
```
# Standard deviation
```

```
sd(response_time)
```

```
# Mean  $\pm$  SD limits
```

```
mean(response_time) - sd(response_time)
```

```
mean(response_time) + sd(response_time)
```



The percentile approach is preferred because it is more appropriate for a skewed response-time distribution and is less dependent on the assumption of normality.

Question 2 – Comparing Many Distributions and Bean Plots

The semiconductor manufacturer measures **chip fabrication cycle time in minutes** for four production lines. The objective is to compare their variability and distribution shape.

Given Data

- **L1:** 42, 44, 45, 46, 47, 48, 49, 50, 51, 53
- **L2:** 38, 41, 44, 47, 50, 53, 56, 59, 62, 65
- **L3:** 45, 46, 46, 47, 47, 48, 48, 49, 49, 50
- **L4:** 40, 41, 42, 43, 44, 45, 48, 55, 61, 72

Task 1 – Create a Visualization Comparing All Four Distributions

R Program

Chip fabrication cycle time data

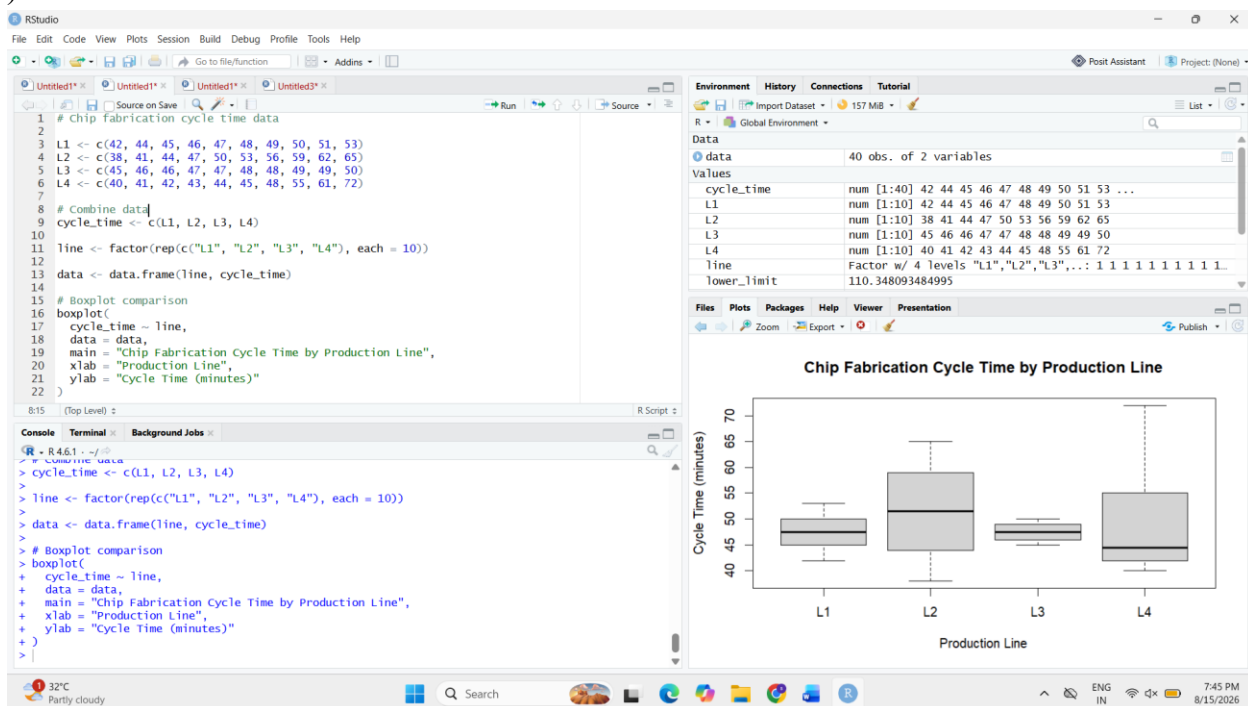
```
L1 <- c(42, 44, 45, 46, 47, 48, 49, 50, 51, 53)
L2 <- c(38, 41, 44, 47, 50, 53, 56, 59, 62, 65)
L3 <- c(45, 46, 46, 47, 47, 48, 48, 49, 49, 50)
L4 <- c(40, 41, 42, 43, 44, 45, 48, 55, 61, 72)
```

```
# Combine data
cycle_time <- c(L1, L2, L3, L4)
```

```
line <- factor(rep(c("L1", "L2", "L3", "L4"), each = 10))
```

```
data <- data.frame(line, cycle_time)
```

```
# Boxplot comparison
boxplot(
  cycle_time ~ line,
  data = data,
  main = "Chip Fabrication Cycle Time by Production Line",
  xlab = "Production Line",
  ylab = "Cycle Time (minutes)"
)
```



Interpretation

The boxplot allows the four production lines to be compared using their central tendency, spread and possible extreme observations.

- **L1** has moderate and relatively consistent variation.
- **L2** has a broad spread across its values.
- **L3** has the smallest spread and values are highly concentrated.
- **L4** has substantial variation, particularly because of the high values **61 and 72 minutes**.

Task 2 – Construct Bean Plots or Equivalent Distribution Visualization Along the Vertical Axis

A **bean plot** displays the distribution of observations together with their density. Since standard R does not provide a built-in bean plot, a **violin plot with individual observations** is used as an equivalent distribution visualization.

R Program

```
# Install ggplot2 if required
```

```
# install.packages("ggplot2")
```

```
library(ggplot2)
```

```
ggplot(data, aes(x = line, y = cycle_time)) +
```

```
  geom_violin(trim = FALSE) +
```

```
  geom_jitter(width = 0.08, size = 2) +
```

```
  labs(
```

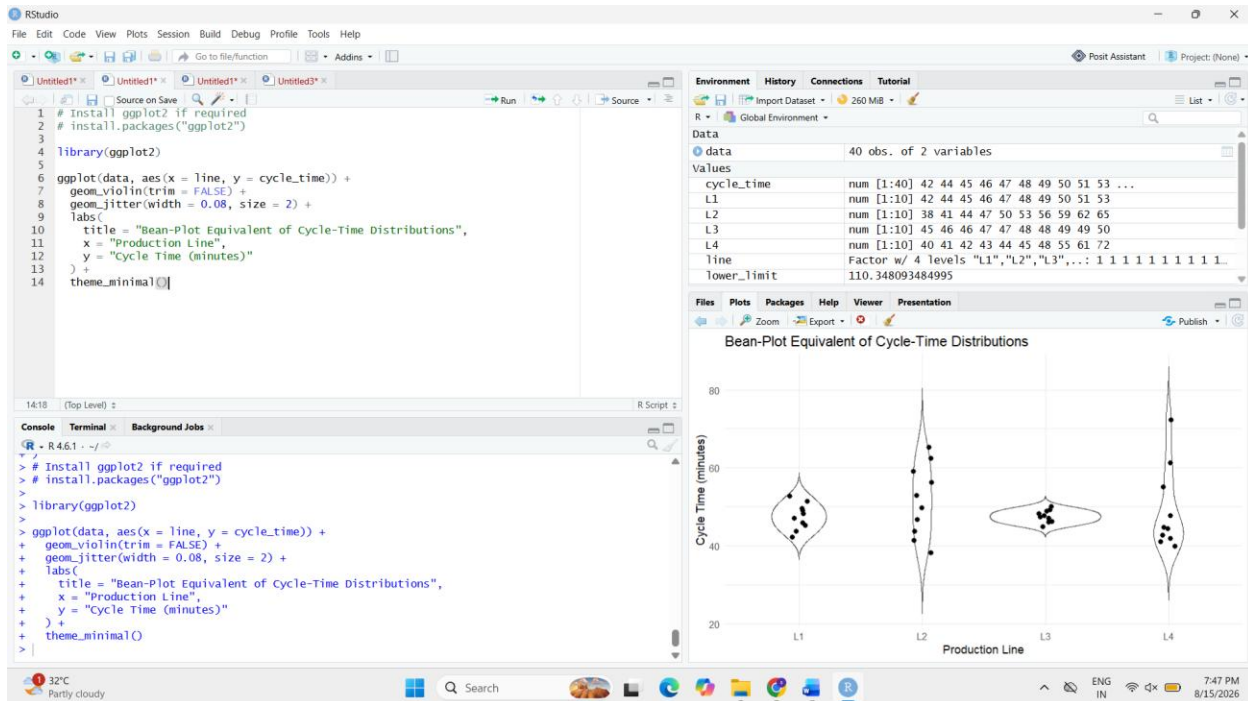
```
    title = "Bean-Plot Equivalent of Cycle-Time Distributions",
```

```
    x = "Production Line",
```

```
    y = "Cycle Time (minutes)"
```

```
  ) +
```

```
  theme_minimal()
```

Interpretation

The vertical-axis distribution visualization clearly shows the shape and concentration of each production line.

- **L3** has a narrow distribution because its observations are concentrated between 45 and 50 minutes.
- **L2** is more widely distributed from 38 to 65 minutes.
- **L4** shows a long upper tail because of values extending to 72 minutes.
- **L1** remains relatively concentrated around the middle range.

Task 3 – Determine Which Line Has Greatest Variability and Which Has the Most Concentrated Distribution

R Program

Calculate standard deviation for each production line

```
sd_values <- c(
```

```
  L1 = sd(L1),
```

```
  L2 = sd(L2),
```

```

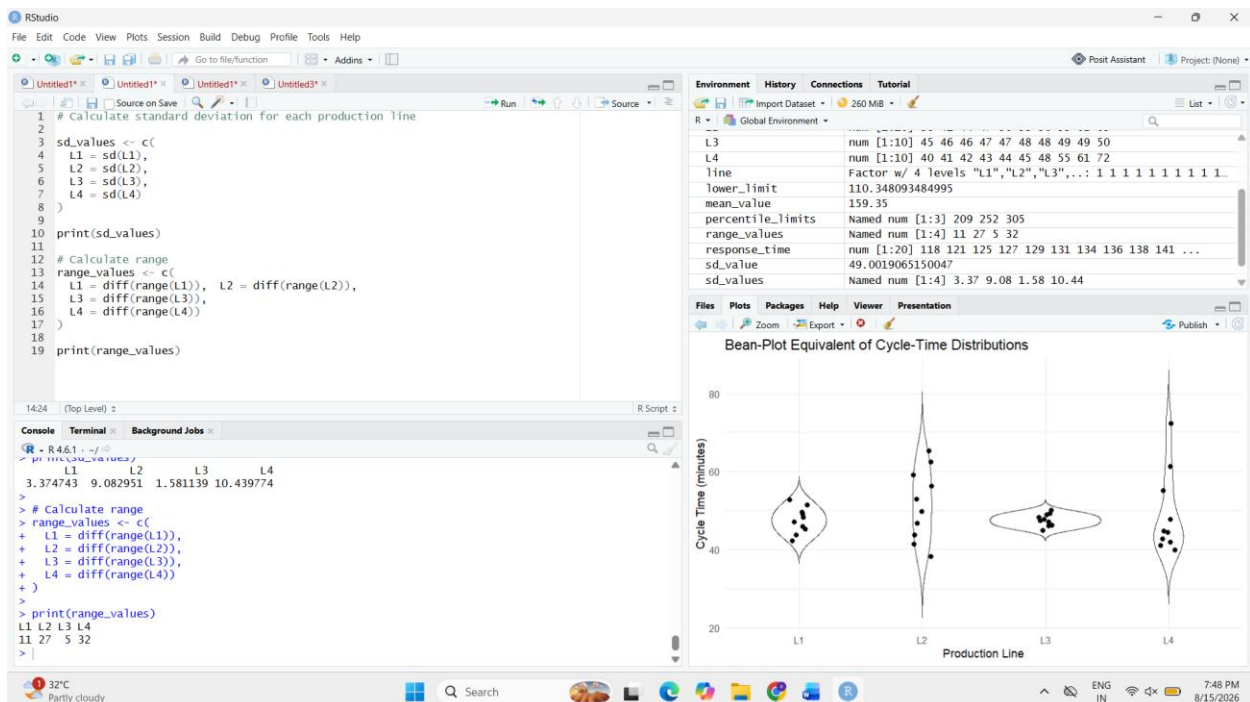
L3 = sd(L3),
L4 = sd(L4)
)

print(sd_values)

# Calculate range
range_values <- c(
  L1 = diff(range(L1)),
  L2 = diff(range(L2)),
  L3 = diff(range(L3)),
  L4 = diff(range(L4))
)

print(range_values)

```



Comparison

Production Line Range (min) Standard Deviation (approx.)

L1	11	3.37
L2	27	9.08
L3	5	1.58
L4	32	11.07

Answer

Greatest variability: L4

L4 has the largest range (**32 minutes**) and the highest standard deviation ($\approx$ **11.07 minutes**). Its values extend from 40 to 72 minutes, showing substantial variation.

Most concentrated distribution: L3

L3 has the smallest range (**5 minutes**) and the lowest standard deviation ($\approx$ **1.58 minutes**). Its observations are tightly concentrated between 45 and 50 minutes.

Therefore:

L4 → Greatest variability

L3 → Most concentrated distribution

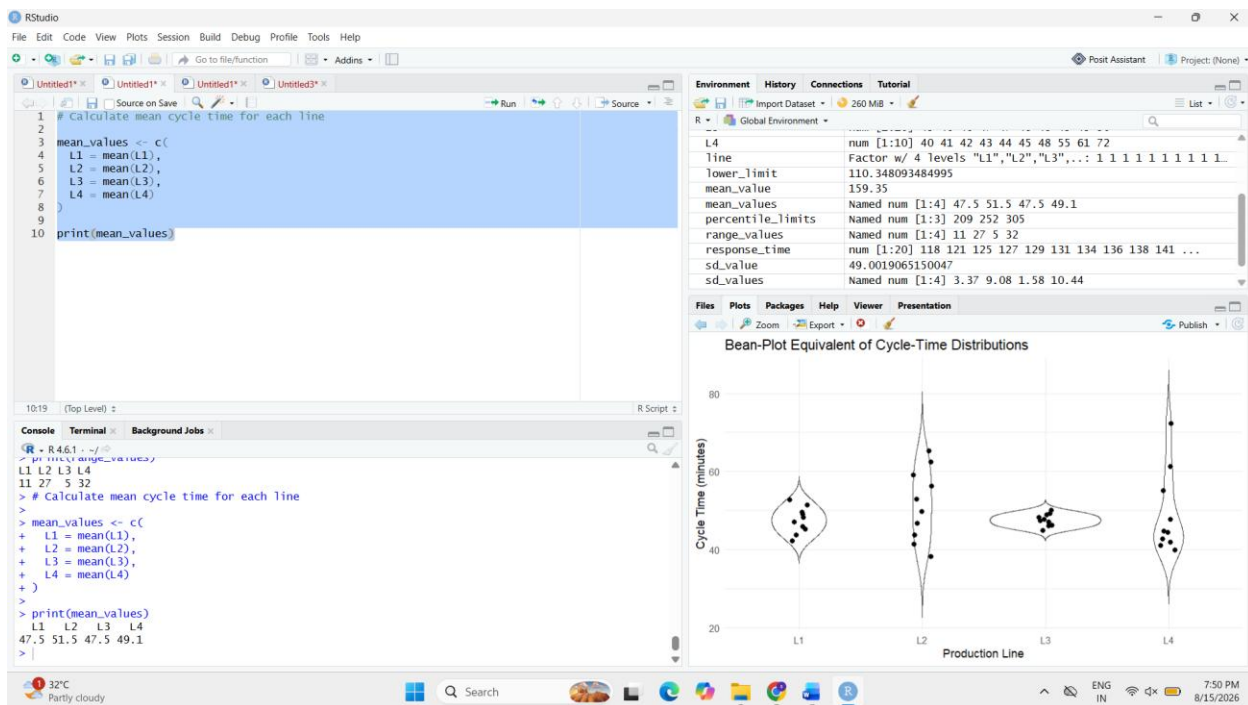
Task 4 – Explain Why Comparing Only Means Is Insufficient

R Program

Calculate mean cycle time for each line

```
mean_values <- c(  
  L1 = mean(L1),  
  L2 = mean(L2),  
  L3 = mean(L3),  
  L4 = mean(L4)  
)
```

```
print(mean_values)
```



Mean Values

Production Line Mean Cycle Time (minutes)

L1	47.5
L2	51.5
L3	47.5
L4	48.1

Explanation

Comparing only the means is insufficient because the mean describes only the **central tendency** and does not show the variability or shape of the distribution.

For example, **L1 and L3 both have a mean of 47.5 minutes**, but their distributions are very different.

- L1 has a range of **11 minutes**.
- L3 has a range of only **5 minutes**.

- L3 is therefore much more concentrated and consistent than L1.

Similarly, L4 has a mean of approximately **48.1 minutes**, which is close to L1 and L3, but L4 has the **greatest variability** because of its high observations.

Therefore, management should consider **mean, standard deviation, range and distribution shape together**, rather than relying only on mean cycle time.

Question 3 – Proportion Visualization and Choice of Chart

The university analyzes the preferred learning mode of **240 students** across CSE, ECE, and IT departments. The required comparison is between **side-by-side bars and a 100% stacked bar chart**, followed by an evaluation of pie charts and misleading design choices.

Given Data

Department Online (%) Blended (%) Face-to-Face (%)

CSE	52	28	20
ECE	45	35	20
IT	30	40	30

Task 1 – Create a Side-by-Side Bar Chart and a 100% Stacked Bar Chart

R Program

```
# Learning mode preference data
```

```
learning_data <- data.frame(
  Department = c("CSE", "ECE", "IT"),
  Online = c(52, 45, 30),
  Blended = c(28, 35, 40),
  Face_to_Face = c(20, 20, 30)
)
```

```
# Convert data to long format
library(ggplot2)
```

```
long_data <- data.frame(
  Department = rep(learning_data$Department, each = 3),
```

```

Learning_Mode = rep(
  c("Online", "Blended", "Face-to-Face"),
  times = 3
),
Percentage = c(
  learning_data$Online,
  learning_data$Blended,
  learning_data$Face_to_Face
)
)

# Side-by-side bar chart
ggplot(long_data,
  aes(x = Department, y = Percentage, fill = Learning_Mode)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(
    title = "Preferred Learning Mode by Department",
    x = "Department",
    y = "Percentage (%)"
  ) +
  theme_minimal()

# 100% stacked bar chart
ggplot(long_data,
  aes(x = Department, y = Percentage, fill = Learning_Mode)) +
  geom_bar(stat = "identity", position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  labs(
    title = "100% Stacked Bar Chart of Learning Preferences",
    x = "Department",
    y = "Proportion"
  ) +
  theme_minimal()

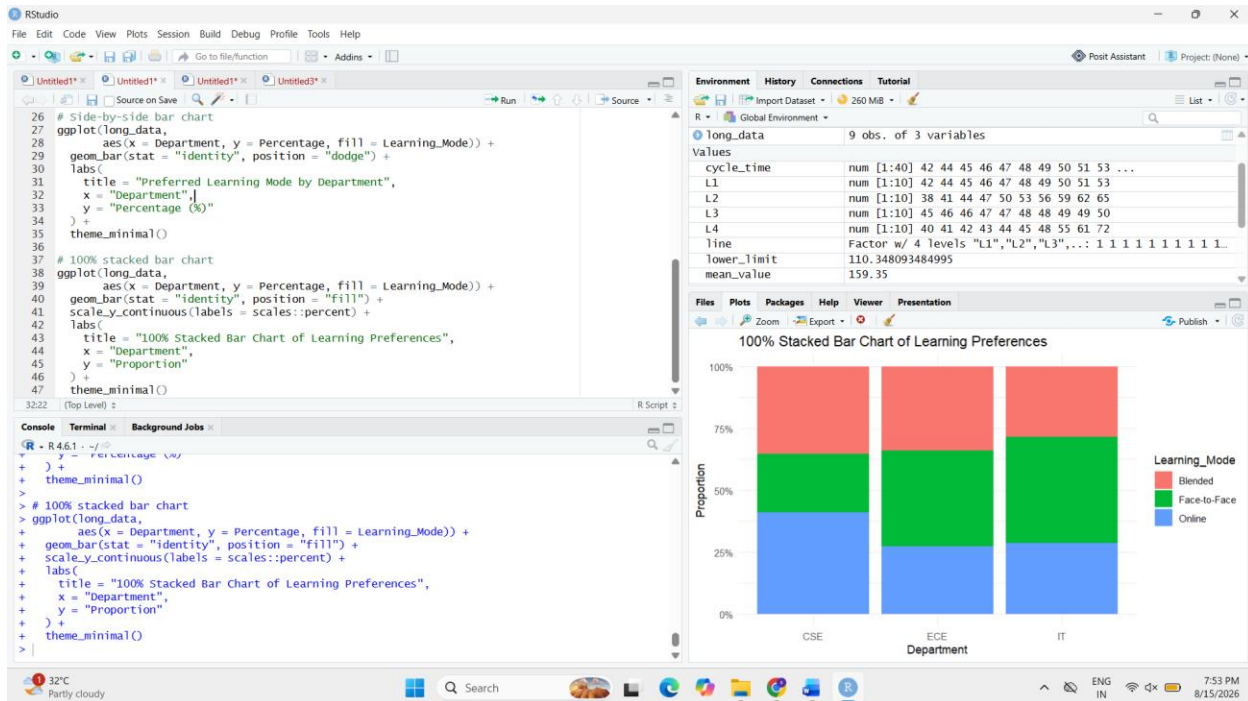
```

Expected Visualizations

You should **run the code in RStudio and include both generated charts** in your submission.

The first graph compares the individual percentages using bars placed side by side.

The second graph represents each department as a complete **100% bar**, divided according to the three learning modes.



Task 2 – Which Chart Better Supports Comparison of Individual Proportions?

Answer

Side-by-side bar chart is better for comparing individual proportions, while the 100% stacked bar chart is better for understanding the overall composition within each department.

Task 3 – Why Are Separate Pie Charts Less Effective?

Separate pie charts are less effective because comparing angles and areas between different pies is more difficult than comparing bars with a common baseline.

For example, comparing:

CSE Online = 52%

with

ECE Online = 45%

is easier using side-by-side bars than trying to visually compare two slices from separate pie charts.

Therefore, multiple pie charts can make small differences harder to identify accurately.

Answer

A side-by-side bar chart provides a **common baseline**, allowing the viewer to compare proportions more directly. Separate pie charts do not provide this same visual reference.

Task 4 – Identify One Design Choice That Could Make Small Proportional Differences Appear Larger

A misleading design choice would be to **truncate the y-axis** of a bar chart instead of starting it at zero.

For example, if the y-axis starts at **40% instead of 0%**, the difference between:

- CSE = 52%
- ECE = 45%

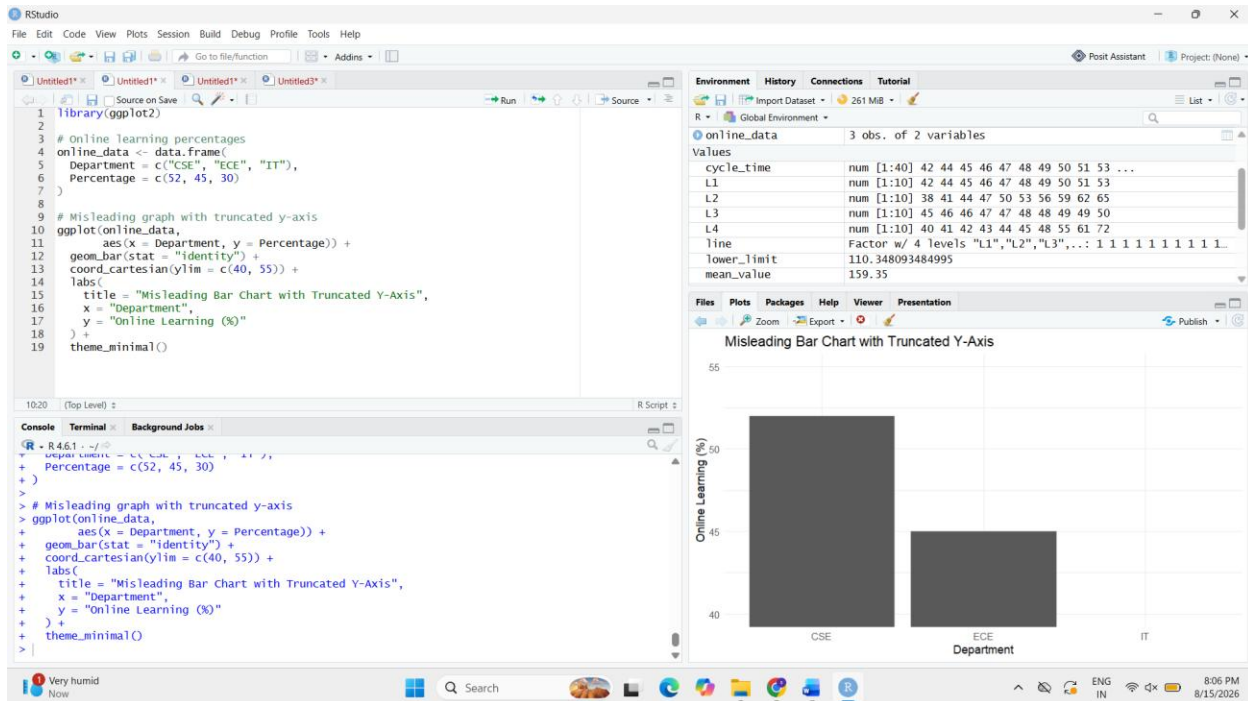
could appear much larger than the actual **7-percentage-point difference**.

R Example of a Potentially Misleading Chart

Potentially misleading truncated-axis chart

```
ggplot(  
  subset(long_data, Learning_Mode == "Online"),  
  aes(x = Department, y = Percentage)  
) +  
  geom_bar(stat = "identity") +  
  coord_cartesian(ylim = c(40, 55)) +  
  labs(  
    title = "Potentially Misleading Truncated Axis",  
    x = "Department",  
    y = "Online Learning (%)"  
  ) +  
  theme_minimal()
```

This graph should be used **only to demonstrate the misleading design**, not as the recommended final visualization.



Why It Is Misleading

The actual difference between CSE and ECE is:

$$52\% - 45\% = 7 \text{ percentage points}$$

But truncating the axis can visually exaggerate this difference.

Question 4 – Nested Proportions using Treemap and Sunburst

The e-commerce company has ₹1.2 crore = ₹120 lakh annual revenue and wants to visualize both category contribution and the internal subcategory composition.

Given Data

Category Subcategory Revenue (₹ lakh)

Electronics	Mobiles	24
Electronics	Laptops	18
Electronics	Accessories	8
Home	Furniture	14

Category	Subcategory	Revenue (₹ lakh)
----------	-------------	------------------

Home	Appliances	10
------	------------	----

Home	Decor	6
------	-------	---

Fashion	Men	7
---------	-----	---

Fashion	Women	8
---------	-------	---

Fashion	Kids	5
---------	------	---

Task 1 – Construct a Treemap

R Program

```
# Install packages if required
```

```
# install.packages("treemap")
```

```
library(treemap)
```

```
revenue_data <- data.frame(
```

```
  Category = c(
```

```
    "Electronics", "Electronics", "Electronics",
```

```
    "Home", "Home", "Home",
```

```
    "Fashion", "Fashion", "Fashion"
```

```
  ),
```

```
  Subcategory = c(
```

```
    "Mobiles", "Laptops", "Accessories",
```

```
    "Furniture", "Appliances", "Decor",
```

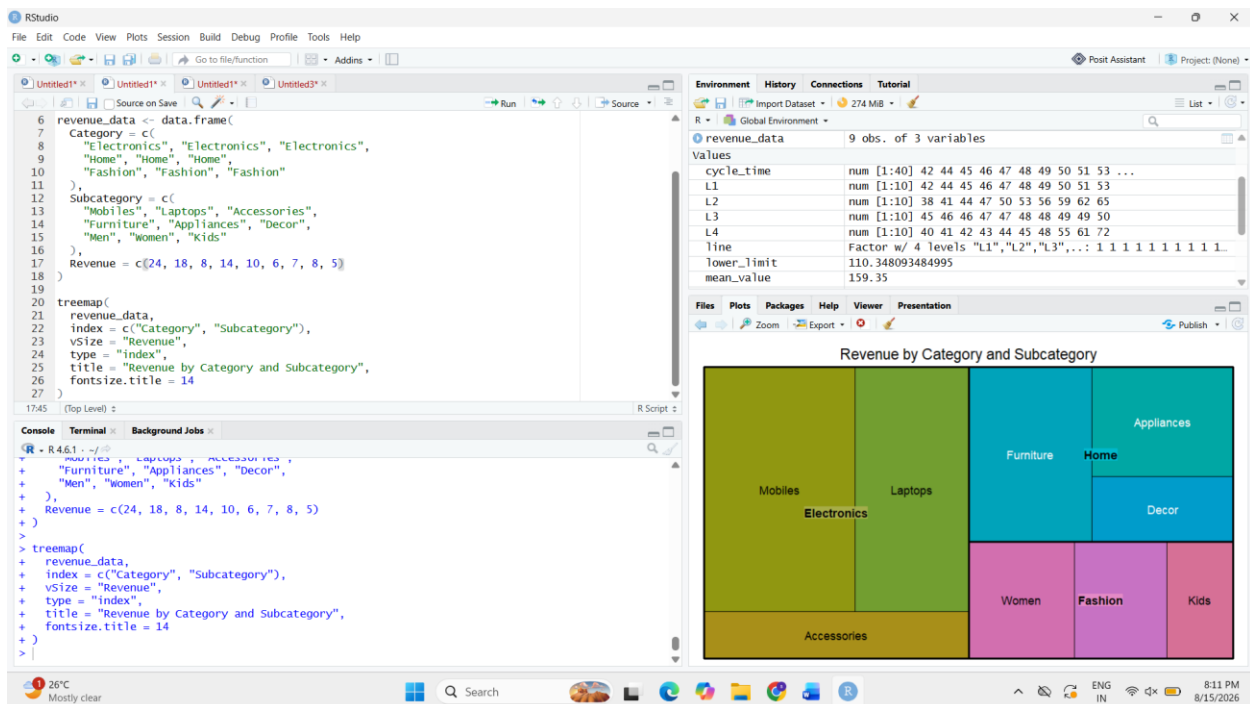
```
    "Men", "Women", "Kids"
```

```
  ),
```

```
  Revenue = c(24, 18, 8, 14, 10, 6, 7, 8, 5)
```

)

```
treemap(  
  revenue_data,  
  index = c("Category", "Subcategory"),  
  vSize = "Revenue",  
  type = "index",  
  title = "Revenue by Category and Subcategory",  
  fontsize.title = 14  
)
```



Interpretation

The treemap represents revenue using **area**.

- Electronics occupies the largest overall area.
- Home occupies the second-largest area.
- Fashion occupies the smallest area.
- Within Electronics, **Mobiles** has the largest subcategory area.

Task 2 – Construct a Sunburst Chart

R Program

```
# Install if required
# install.packages("plotly")

library(plotly)

sunburst_data <- data.frame(
  Category = c(
    "Electronics", "Electronics", "Electronics",
    "Home", "Home", "Home",
    "Fashion", "Fashion", "Fashion"
  ),
  Subcategory = c(
    "Mobiles", "Laptops", "Accessories",
    "Furniture", "Appliances", "Decor",
    "Men", "Women", "Kids"
  ),
  Revenue = c(24, 18, 8, 14, 10, 6, 7, 8, 5)
)

plot_ly(
  data = sunburst_data,
  ids = ~Subcategory,
  labels = ~Subcategory,
  parents = ~Category,
  values = ~Revenue,
```

```

type = "sunburst",

branchvalues = "total"

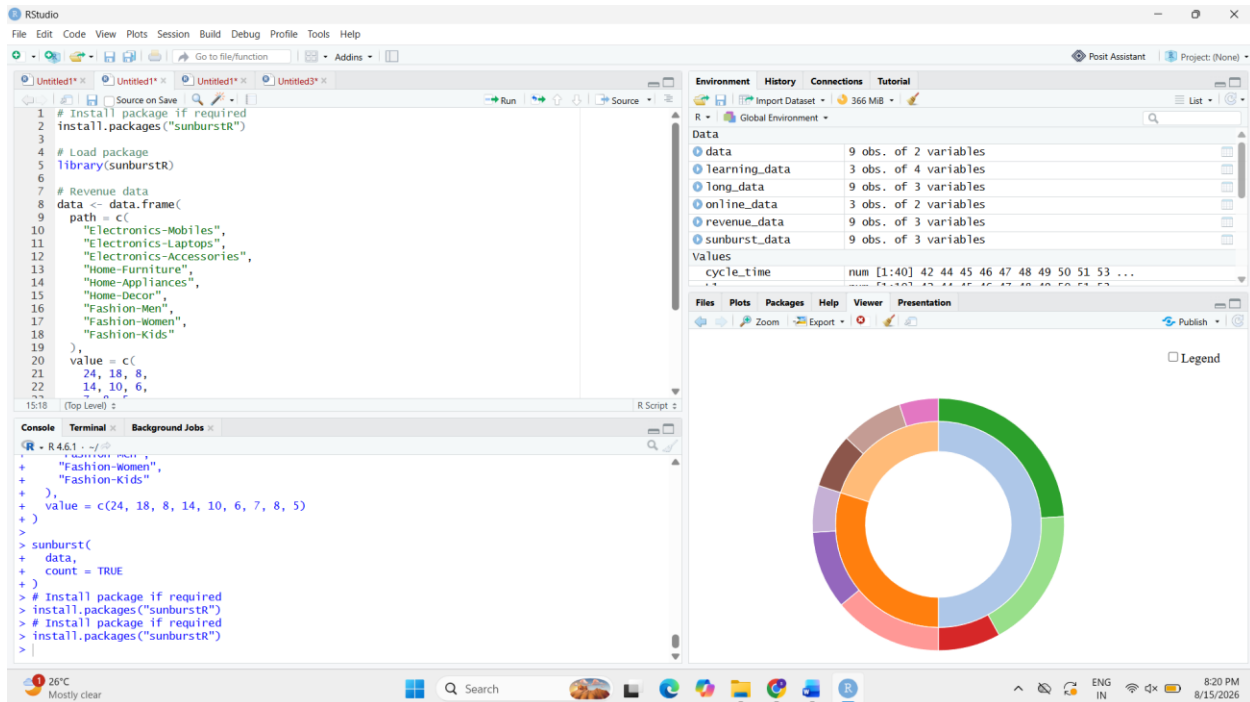
)%>%

layout(

  title = "Sunburst Chart of Revenue"

)

```



Interpretation

The sunburst chart represents the same hierarchy using concentric levels:

Centre → Category → Subcategory

The size of each segment represents its revenue contribution.

Task 3 – Determine the Largest Category and Calculate Its Percentage of Total Revenue

R Program

```
# Calculate category totals
```

```
category_total <- aggregate(
  Revenue ~ Category,
```

```
data = revenue_data,
sum
)

print(category_total)

# Total revenue
total_revenue <- sum(revenue_data$Revenue)

# Largest category
largest_category <- category_total[
  which.max(category_total$Revenue),
]

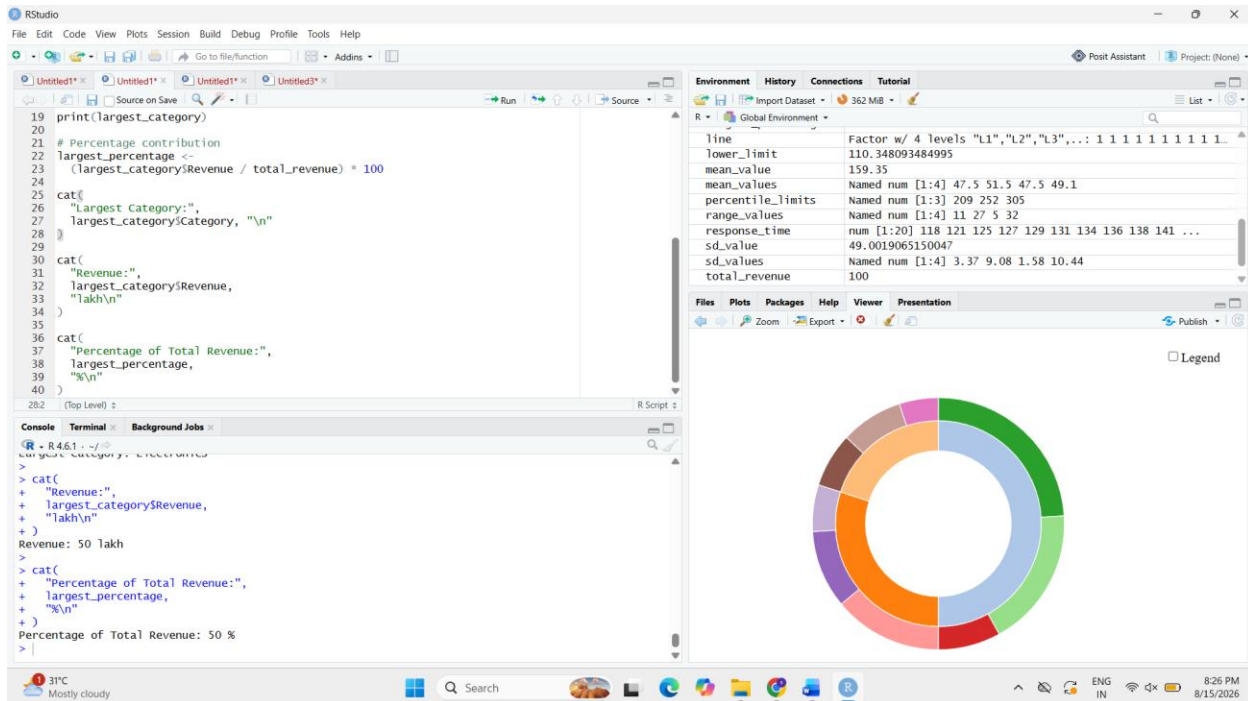
print(largest_category)

# Percentage contribution
largest_percentage <-
  (largest_category$Revenue / total_revenue) * 100

cat(
  "Largest Category:",
  largest_category$Category, "\n"
)

cat(
  "Revenue:",
  largest_category$Revenue,
  "lakh\n"
)

cat(
  "Percentage of Total Revenue:",
  largest_percentage,
  "%\n"
)
```



Calculation

Electronics:

$$24 + 18 + 8 = \text{₹}50 \text{ lakh}$$

Total revenue represented by the given data:

$$50 + 30 + 20 = \text{₹}100 \text{ lakh}$$

Therefore:

$$\text{Electronics percentage} = (50 / 100) \times 100 = 50\%$$

Answer

Largest category: Electronics

Revenue: ₹50 lakh

Percentage: 50% of the revenue represented in the supplied category data.

Note: The scenario states annual revenue of ₹1.2 crore (₹120 lakh), but the listed subcategory values add up to **₹100 lakh**, not ₹120 lakh. Therefore, using the supplied table, Electronics is **50% of the listed revenue**.

Task 4 – Compare Treemap and Sunburst for Identifying Nested Proportions

Treemap

A treemap represents categories and subcategories using **nested rectangular areas**.

Advantages:

- Makes large revenue contributions easy to identify.
- Efficiently uses available space.
- Allows category and subcategory sizes to be compared simultaneously.

Limitation:

- Comparing areas of similarly sized rectangles can sometimes be difficult.

Sunburst

A sunburst represents the hierarchy using **concentric rings**.

Advantages:

- Clearly shows the parent-child relationship.
- Makes the hierarchy between category and subcategory easy to understand.
- Useful for exploring multiple hierarchical levels.

Limitation:

- Comparing the sizes of arc segments can be harder, especially for small segments.

Comparison

Feature	Treemap	Sunburst
Category contribution	Very clear	Clear
Subcategory contribution	Clear	Clear
Hierarchy	Moderate	Very clear
Area comparison	Better	More difficult
Nested structure	Good	Excellent

Conclusion

The **treemap is better for comparing revenue magnitude**, while the **sunburst is better for understanding the hierarchical relationship between categories and subcategories**.

Task 5 – State One Situation in Which Area Encoding Could Mislead Interpretation

Area encoding can be misleading when **two regions have similar areas but their shapes are very different**.

For example, a long, thin rectangle and a nearly square rectangle may have similar areas, but the viewer may perceive the larger-looking shape as having greater revenue.

Another issue occurs when several small areas are placed next to a very large area. The small differences between them can become difficult to perceive accurately.

Answer

Area encoding can mislead when users compare similarly sized regions or regions with very different shapes. Therefore, a treemap should be used mainly for identifying broad differences in contribution rather than making highly precise numerical comparisons.

Question 5 – Flow Proportions using Sankey Diagram / Parallel Sets

A smart-city mobility study follows **200 commuters** from residential zone → primary transport mode → destination zone. The planning team wants to identify dominant flows and possible bottlenecks.

Given Data

Origin	Mode	Destination	Commuters
--------	------	-------------	-----------

Residential	Bus	Central	35
-------------	-----	---------	----

Residential	Bus	Industrial	15
-------------	-----	------------	----

Residential	Train	Central	25
-------------	-------	---------	----

Residential	Train	Industrial	10
-------------	-------	------------	----

Suburban	Bus	Central	20
----------	-----	---------	----

Suburban	Bus	Industrial	10
----------	-----	------------	----

Suburban	Train	Central	30
----------	-------	---------	----

Suburban	Train	Industrial	15
----------	-------	------------	----

Rural	Bus	Central	12
-------	-----	---------	----

Origin	Mode	Destination	Commuters
--------	------	-------------	-----------

Rural	Bus	Industrial	8
-------	-----	------------	---

Rural	Train	Central	10
-------	-------	---------	----

Rural	Train	Industrial	10
-------	-------	------------	----

Task 1 – Construct a Sankey Diagram

R Program

```
# Install packages if required
```

```
install.packages("networkD3")
```

```
install.packages("dplyr")
```

```
library(networkD3)
```

```
library(dplyr)
```

```
# Given data
```

```
data <- data.frame(
```

```
  Origin = c(
```

```
    "Residential", "Residential", "Residential", "Residential",
```

```
    "Suburban", "Suburban", "Suburban", "Suburban",
```

```
    "Rural", "Rural", "Rural", "Rural"
```

```
),
```

```
  Mode = c(
```

```
    "Bus", "Bus", "Train", "Train",
```

```
    "Bus", "Bus", "Train", "Train",
```

```
    "Bus", "Bus", "Train", "Train"
```

```
),
```

```
Destination = c(
  "Central", "Industrial", "Central", "Industrial",
  "Central", "Industrial", "Central", "Industrial",
  "Central", "Industrial", "Central", "Industrial"
),
Commuters = c(
  35, 15, 25, 10,
  20, 10, 30, 15,
  12, 8, 10, 10
)
)
```

```
# Create nodes
nodes <- data.frame(
  name = unique(c(
    data$Origin,
    data$Mode,
    data$Destination
  ))
)
```

```
# Create links
links <- data.frame(
  source = match(data$Origin, nodes$name) - 1,
  target = match(data$Mode, nodes$name) - 1,
  value = data$Commuters
)
```

```
# Add Mode -> Destination links

links2 <- data.frame(
  source = match(data$Mode, nodes$name) - 1,
  target = match(data$Destination, nodes$name) - 1,
  value = data$Commuters
)
```

```
# Combine links

all_links <- rbind(links, links2)
```

```
# Sankey diagram

sankeyNetwork(
  Links = all_links,
  Nodes = nodes,
  Source = "source",
  Target = "target",
  Value = "value",
  NodeID = "name",
  fontSize = 13,
  nodeWidth = 30
)
```

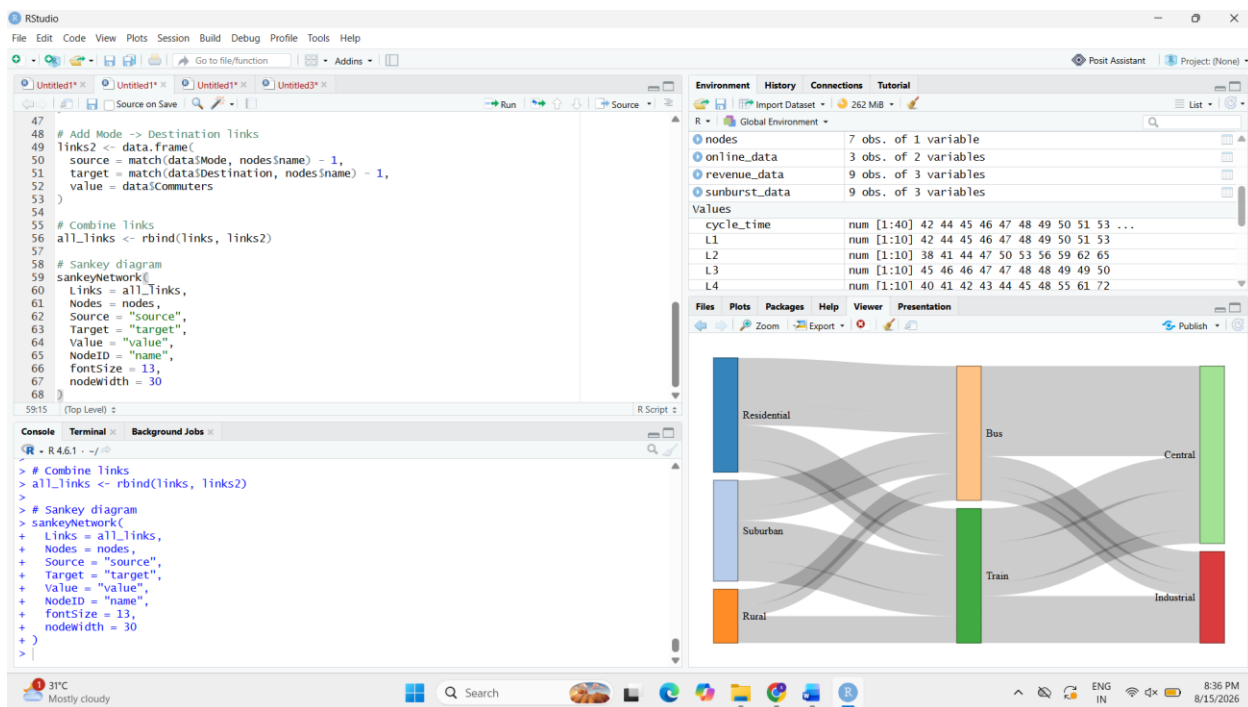
Interpretation

The Sankey diagram represents the flow:

Origin → Transport Mode → Destination

The width of each link represents the number of commuters following that particular flow. Wider links therefore indicate larger commuter movements.

The visualization makes it easier to identify dominant routes and potential transport bottlenecks.



Task 2 – Construct a Parallel-Sets Representation

R Program

```
# Install package if required
```

```
install.packages("ggparallel")
```

```
library(ggplot2)
```

```
library(ggparallel)
```

```
# Parallel sets representation
```

```
ggplot(
```

```
  data,
```

```
  aes(
```

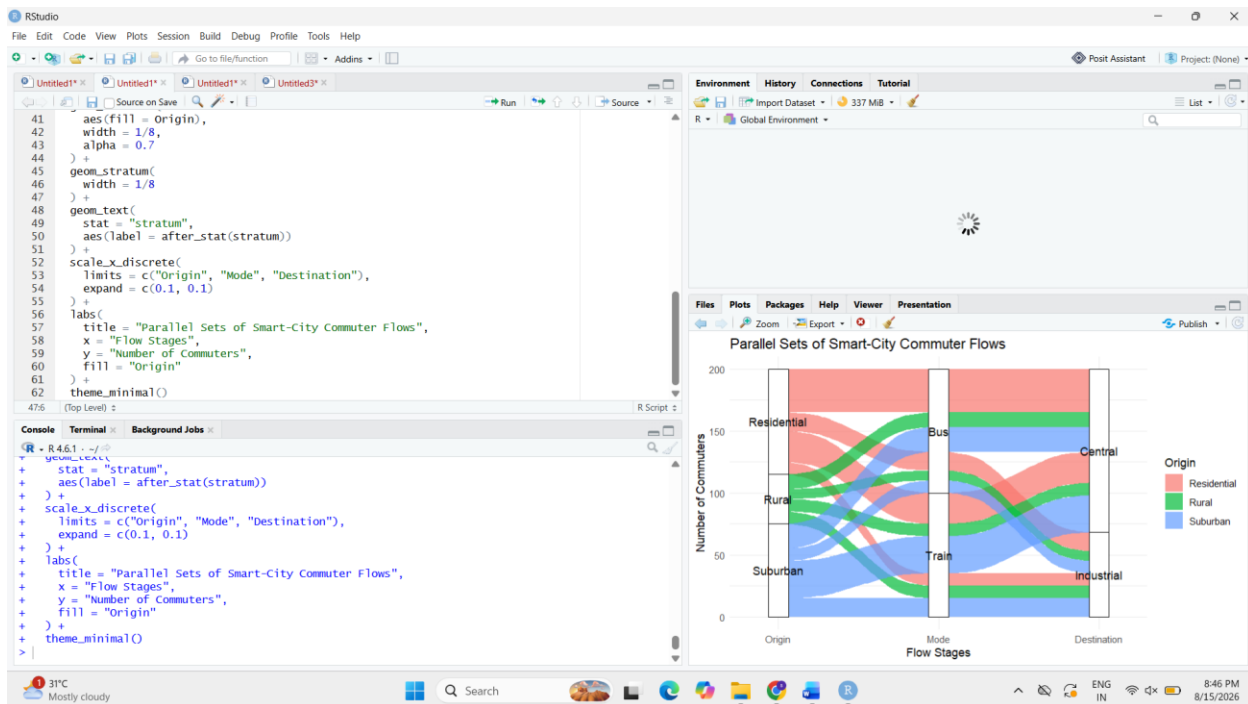
```
    x = Origin,
```

```
    y = Commuters,
```

```

    fill = Mode
  )
) +
  geom_parallel_sets(alpha = 0.7) +
  geom_parallel_sets_axes() +
  geom_parallel_sets_labels() +
  labs(
    title = "Parallel Sets of Smart-City Commuter Flows",
    x = "Origin",
    y = "Number of Commuters"
  ) +
  theme_minimal()

```



Interpretation

The parallel-sets representation shows relationships between the categorical variables:

Origin → Mode → Destination

The thickness of the connecting bands represents commuter volume. It allows the relationships between origins, transport modes, and destinations to be compared simultaneously.

Task 3 – Identify the Three Largest Flows and Calculate Their Percentages

The three largest individual flows in the given dataset are:

1. **Residential $\rightarrow$ Bus $\rightarrow$ Central = 35 commuters**
2. **Suburban $\rightarrow$ Train $\rightarrow$ Central = 30 commuters**
3. **Residential $\rightarrow$ Train $\rightarrow$ Central = 25 commuters**

R Program

Find the three largest flows

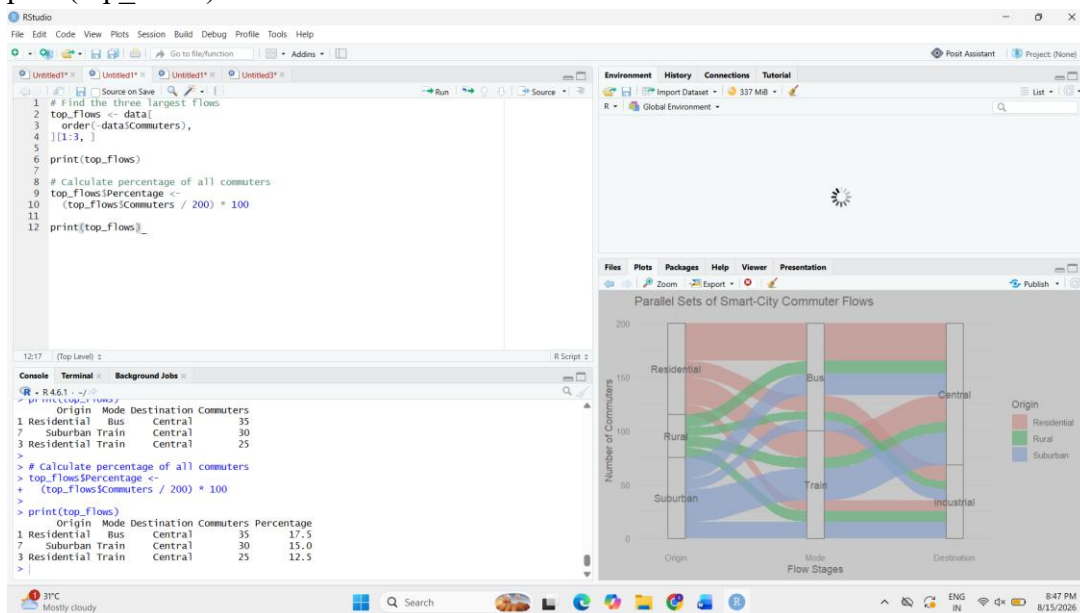
```
top_flows <- data[
  order(-data$Commuters),
][1:3, ]
```

```
print(top_flows)
```

```
# Calculate percentage of all commuters
```

```
top_flows$Percentage <-  
(top_flows$Commuters / 200) * 100
```

```
print(top_flows)
```



Calculation

Flow	Commuters	Percentage
Residential → Bus → Central	35	17.5%
Suburban → Train → Central	30	15%
Residential → Train → Central	25	12.5%

Answer

The three largest flows account for:

$$17.5\% + 15\% + 12.5\% = 45\%$$

Therefore, the three largest individual flows represent **45% of all commuters**.

Task 4 – Compare Sankey and Parallel Sets

Sankey Diagram

A Sankey diagram is particularly effective for showing **flow magnitude**.

The width of each connection directly represents the number of commuters. Therefore, large flows such as **Residential → Bus → Central (35 commuters)** are immediately visible.

Parallel Sets

Parallel sets are more focused on showing **relationships between multiple categorical dimensions**.

They are useful for seeing how commuters move from:

Origin → Mode → Destination

and for understanding the overall categorical structure.

Comparison

Feature	Sankey	Parallel Sets
Flow magnitude	Excellent	Good
Categorical relationships	Good	Excellent
Identifying dominant flows	Very easy	Easy

Feature	Sankey	Parallel Sets
Multiple categorical dimensions	Good	Very good
Bottleneck identification	Very useful	Useful

Conclusion

The **Sankey diagram** is better for identifying the magnitude of individual flows, while the **parallel-sets representation** is useful for examining relationships between multiple categorical variables.

Task 5 – Explain How Poor Link Widths or Category Ordering Could Mislead the Viewer

A Sankey diagram can become misleading if the link widths do not accurately represent the flow values.

For example, if a link representing 10 commuters were drawn almost as thick as a link representing 35 commuters, viewers could incorrectly assume that the two flows are similar.

Similarly, poor category ordering can cause links to cross excessively. This can make it difficult to follow a particular flow and may cause the viewer to misunderstand the relationships.

Answer

Two possible sources of misleading interpretation are:

- **Incorrect link widths:** make flow magnitudes appear larger or smaller than their actual values.
- **Poor category ordering:** creates excessive crossing of links, making individual flows difficult to follow.

Therefore, link widths should be **proportional to commuter counts**, and categories should be ordered to maintain a clear flow structure.

S. LAKSHITA

192524108